PROYECTO INTERMODULAR II — 2.º DAW
Documento 06: Digitalización y Sostenibilidad
Alumno: Damián Conejos
Caso: Panel de Gestión de Entradas y Aforos de TechFest

1. Mejora en la digitalización del evento

- **Cómo se hace ahora en Attendize v2.8.0:** Si durante el TechFest un taller se llena y el coordinador necesita pausar la venta o cambiar el aforo, el panel actual tarda varios segundos en responder, recarga trozos de página y a veces el script de jQuery se queda colgado.
- **Cómo mejora con la modernización:** Al usar React e Inertia.js, el cambio de estado (pausar o reanudar) o la edición del aforo se hacen al instante desde un portátil o el móvil, sin parpadeos de pantalla y con comprobación inmediata de los datos antes de guardarlos en MySQL.

Dos indicadores comparables

1. **Tiempo para actualizar un ticket:**
    - *Antes (Attendize v2.8.0):* Unos **3 segundos** por cada acción (abrir modal, esperar respuesta del servidor y recargar la vista).
    - *Ahora (Modernizado):* Menos de **0,5 segundos** (la petición asíncrona solo actualiza los datos que han cambiado).
    - *Cómo se mide:* Con la pestaña **Red (Network)** de las herramientas de desarrollador del navegador.
2. **Formularios enviados con errores al servidor:**
    - *Antes:* Alrededor de un **15% - 20%** de los envíos daban fallo porque el usuario solo se enteraba de un error (como dejar el nombre vacío o poner aforo negativo) después de enviarlo al servidor.
    - *Ahora:* **Menos del 2%**, ya que React y TypeScript avisan en rojo al momento en el propio formulario y no dejan pulsar «Guardar» si hay datos inválidos.
    - *Cómo se mide:* Contando los errores de validación (código 422) en el registro de Laravel (storage/logs/laravel.log).

2. Sostenibilidad y ahorro de recursos

Para que la aplicación consuma menos recursos técnicos y energéticos en el servidor y en los dispositivos móviles, se aplican dos medidas sencillas:

Medida 1: Reducción del tráfico de red (menos datos transferidos)

- **Qué se hace:** En lugar de enviar bloques enteros de código HTML desde el servidor con Blade cada vez que se toca un botón, con Inertia.js solo se envían pequeños datos en formato JSON con la información imprescindible.
- **Ahorro:** La red WiFi del evento no se satura y los móviles gastan menos datos y batería.
- **Cómo se mide:** Mirando el tamaño de transferencia en la pestaña **Red (Network)** del navegador. Una respuesta con HTML de Blade pesa unos **40 KB**, mientras que el JSON de Inertia ocupa **menos de 2 KB** (un ahorro superior al 90%).

Medida 2: Reducción de consultas a MySQL (menos uso de procesador)

- **Qué se hace:** En lugar de lanzar una consulta SQL por cada ticket para contar cuántas plazas se han vendido (el clásico problema de consultas N+1 de Attendize antiguo), en Laravel se agrupan las consultas en una sola sentencia optimizada.
- **Ahorro:** El servidor de base de datos trabaja mucho menos tiempo, reduciendo el consumo de procesador (CPU) y la energía del servidor.
- **Cómo se mide:** Revisando el número de consultas que se ejecutan en MySQL al abrir la pantalla: se pasa de ejecutar **más de 8 consultas** a solo **2 consultas directas**.